

Ofuscação de código em arquivos ELF utilizando relocação

João C. Fukuda

Cryptorave

2026

whoami



- 



ASM

Conteúdo

head

- Motivação
- ELF Refresher
- A Ideia
- Desenvolvimento
- Problemas e Limitações
- Detecção

Motivação

hostname

ARQUIVOS
ELF

INFECÇÃO

VX

MALWARE

- Inspirações:
 - Netspooky's ELF Binary Mangling Series[1]
 - sbliip's Implementing the PT_NOTE Infection Method In x64 Assembly[2]
 - s01den's A short note on entrypoint obscuring in ELF binaries[3]
 - S0S4's Relocation Revelation: Unpacking the Secrets of ELF[4]
- Curiosidade & Conhecimento

DYNAMIC

OFUSCAÇÃO

RELOCAÇÃO

Quick ELF Refresher

info

- A estrutura de um arquivo ELF[5]

- Elf Header (Elf64_Ehdr)

- ▶ e_type, e_entry
 - ▶ e_phoff, e_phentsize, e_phnum
 - ▶ e_shoff, e_shentsize, e_shnum

- Program Header (Elf64_Phdr)

- ▶ Loadable program segment (PT_LOAD)

- Section Header (Elf64_Shdr)

- ▶ Relocation entries with addends (SHT_RELA)
 - ▶ Relocation entries, no addends (SHT_REL)

Phdr[]
Shdr[]

A Ideia

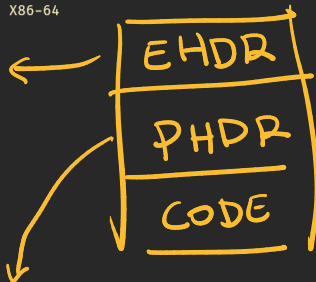
cat

- **Cenário:** Construir um arquivo ELF do zero
 - começar com um arquivo ELF simples e funcional
- **Objetivo:** Fazer a relocação dinâmica funcionar
 - iterar sobre o arquivo até obter algum resultado
- **Conclusão:** Esconder um código na sessão de relocação
 - fazer rodar outro código do que está presente no segmento PT_LOAD

RELA

ELF Header:

```
Magic:  7f 45 4c 46 02 01 01 03 00 00 00 00 00 00 00 00
Class:                                     ELF64
Data:                                       2's complement, little endian
Version:                                  1 (current)
OS/ABI:                                    UNIX - GNU
ABI Version:                              0
• Type: EXEC (Executable file)
Machine: Advanced Micro Devices X86-64
Version: 0x1
• Entry point address: 0x42078
Start of program headers: 64 (bytes into file)
Start of section headers: 0 (bytes into file)
Flags: 0x0
Size of this header: 64 (bytes)
Size of program headers: 56 (bytes)
• Number of program headers: 1
Size of section headers: 64 (bytes)
Number of section headers: 0
Section header string table index: 0
```



There are no sections in this file.

There are no section groups in this file.

Program Headers:

Type	Offset	VirtAddr	PhysAddr
	FileSiz	MemSiz	Flags Align
<u>LOAD</u>	0x0000000000000078	0x00000000000042078	0x00000000000042078
	0x0000000000000032	0x0000000000000032	R E 0x1000

There is no dynamic section in this file.

There are no relocations in this file.

No processor specific unwind information to decode

Dynamic symbol information is not available for displaying symbols.

No version information found in this file.

There is no GOT section in this file.

```
rela (git: main) $ ./relocation_base  
Hello, world!
```



```
rela (git: main) $ cat evil.asm
bits 64
_start:
    int3
rela (git: main) $ objdump -Mintel -j.text -d ./evil

./evil:      file format elf64-x86-64
```

Disassembly of section .text:

```
0000000000401000 <_start>:
    401000:    cc                ← int3
rela (git: main) $ ./evil
Trace/breakpoint trap      (core dumped) ./evil
```

Construindo um ELF do zero

Runtime Linker

When creating a dynamic object that initiates dynamic linking, the link-editor adds a program header element of type `PT_INTERP` to an executable file. This element instructing the system to invoke the runtime linker as the program interpreter: [exec\(2\)](#) and the runtime linker cooperate to create the process image for the program.

The link-editor constructs various data for executable and shared object files that assist the runtime linker. These data reside in loadable segments, making them available during execution. These segments include:

- A `.dynamic` section with type `SHT_DYNAMIC` that holds various data. The structure residing at the beginning of the section holds the addresses of other dynamic linking information.
- The `.got` and `.plt` sections with type `SHT_PROGBITS` that hold two separate tables: the global offset table and the procedure linkage table. Sections below explain how the runtime linker uses and changes the tables to create memory images for object files.

PT_INTERP

Specifies the location and size of a null-terminated path name to invoke as an interpreter. This segment type is mandatory for dynamic executable files and can occur in shared objects. It cannot occur more than once in a file. This type, if present, it must precede any loadable segment entry. See "[Program Interpreter](#)" for further information.

```
rela (git: main) $ ldd /bin/true
linux-vdso.so.1 (0x00007f5046fdb000)
libc.so.6 ⇒ /usr/lib/libc.so.6 (0x00007f5046c00000)
/lib64/ld-linux-x86-64.so.2 ⇒ /usr/lib64/ld-linux-x86-64.so.2 (0x00007f5046fdd000)
```

Construindo do Zero

code

executavel → dinâmico

- modificar o tipo do arquivo de ET_EXEC para ET_DYN
- adicionar o segmento PT_INTERP
- adicionar a sessão SHT_REL

→ interpretador

↓
relocação

```
rela (git: main) $ readelf -lSr relocation_base
There is 1 section header, starting at offset 0xfe:
```

Section Header:

[Nr]	Name	Type	Address	Offset
	Size	EntSize	Flags Link Info Align	
[0]	<no-strings>	RELA	0000000000000000	0000013e
	0000000000000018	0000000000000018	0 0	0

} SHDRs

Key to Flags:

W (write), A (alloc), X (execute), M (merge), S (strings), I (info),
L (link order), O (extra OS processing required), G (group), T (TLS),
C (compressed), x (unknown), o (OS specific), E (exclude),
R (retain), D (mbind), l (large), p (processor specific)

Elf file type is DYN (Shared object file)

Entry point 0x420cc

There are 2 program headers, starting at offset 64

Program Headers:

Type	Offset	VirtAddr	PhysAddr
	FileSiz	MemSiz	Flags Align
<u>INTERP</u>	0x00000000000000b0	0x000000000000420b0	0x000000000000420b0
	0x000000000000001c	0x000000000000001c	R 0x1
	[Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]		
LOAD	0x00000000000000cc	0x000000000000420cc	0x000000000000420cc
	0x0000000000000032	0x0000000000000032	R E 0x1000

interpreter

Relocation section 0 at offset 0x13e contains 1 entry:

Offset	Info	Type	Sym. Value	Sym. Name + Addend
00000000420cc	00000000001	R_X86_64_64		cc

→

RELA

tipo

adendo

```
rela (git: main) $ ./relocation_base  
Segmentation fault (core dumped) ./relocation_base
```



```
[139] rela (git: main) $ valgrind ./relocation_base
=148832= Memcheck, a memory error detector
=148832= Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
=148832= Using Valgrind-3.25.1 and LibVEX; rerun with -h for copyright info
=148832= Command: ./relocation_base
=148832=
Inconsistency detected by ld.so: rtld.c: 1236: rtld_setup_main_map: Assertion `_dl_rtlld_map.l_libname' failed!
=148832=
=148832= HEAP SUMMARY:
=148832=   in use at exit: 0 bytes in 0 blocks
=148832=   total heap usage: 0 allocs, 0 frees, 0 bytes allocated
=148832=
=148832= All heap blocks were freed -- no leaks are possible
=148832=
=148832= For lists of detected and suppressed errors, rerun with: -s
=148832= ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

```
1220  /* Adjust the address of the TLS initialization image in case
1221     the executable is actually an ET_DYN object.  */
1222  if (main_map->l_tls_initimage != NULL)
1223     main_map->l_tls_initimage
1224     = (char *) main_map->l_tls_initimage + main_map->l_addr;
1225  if (! main_map->l_map_end)
1226     main_map->l_map_end = ~0;
1227  if (! _dl_rtld_map.l_libname && _dl_rtld_map.l_name)
1228     {
1229         /* We were invoked directly, so the program might not have a
1230            PT_INTERP.  */
1231         _dl_rtld_libname.name = _dl_rtld_map.l_name;
1232         /* _dl_rtld_libname.next = NULL; Already zero.  */
1233         ? _dl_rtld_map.l_libname = &_dl_rtld_libname;
1234     }
1235  else
1236     assert (_dl_rtld_map.l_libname); /* How else did we get here?  */
1237
1238  return has_interp;
1239 }
```

```
1121  /* Scan the program header table for the dynamic section. */
1122  for (const ElfW(Phdr) *ph = phdr; ph < &phdr[phnum]; ++ph)
1123      switch (ph->p_type)
1124      {
1125          case PT_PHDR:
1126              /* Find out the load address. */
1127              main_map->l_addr = (ElfW(Addr)) phdr - ph->p_vaddr;
1128              break;
1129          case PT_DYNAMIC:
1130              /* This tells us where to find the dynamic section,
1131                 which tells us everything we need to do. */
1132              main_map->l_ld = (void *) main_map->l_addr + ph->p_vaddr;
1133              main_map->l_ld_readonly = (ph->p_flags & PF_W) == 0;
1134              break;
1135          case PT_INTERP:
1136              /* This "interpreter segment" was used by the program loader to
1137                 find the program interpreter, which is this program itself, the
1138                 dynamic linker. We note what name finds us, so that a future
1139                 dlopen call or DT_NEEDED entry, for something that wants to link
1140                 against the dynamic linker as a shared library, will know that
1141                 the shared object is already loaded. */
1142              _dl_rtl_d_libname.name = ((const char *) main_map->l_addr
1143                  + ph->p_vaddr);
1144              /* _dl_rtl_d_libname.next = NULL;          Already zero. */
1145              _dl_rtl_d_map.l_libname = &_dl_rtl_d_libname;
1146
1147              has_interp = true;
1148              break;
```

Aqui!

PT_PHDR

Specifies the location and size of the program header table itself, both in the file and in the memory image of the program. This segment type cannot occur more than once in a file. Moreover, it can occur only if the program header table is part of the memory image of the program. This type, if present, must precede any loadable segment entry. See "[Program Interpreter](#)" for further information.

Construindo do Zero

code

- adicionar o segmento PT_PHDR
- modificar o alcance do segmento PT_LOAD para incorporar os Phdrs

```
rela (git: main) $ readelf -lSr ./relocation_base
There is 1 section header, starting at offset 0x136:
```

Section Header:

[Nr]	Name	Type	Address	Offset
	Size	EntSize	Flags Link Info	Align
[0]	<no-strings>	RELA	0000000000000000	00000176
	0000000000000018	0000000000000018	0 0 0	0

Key to Flags:

W (write), A (alloc), X (execute), M (merge), S (strings), I (info),
L (link order), O (extra OS processing required), G (group), T (TLS),
C (compressed), x (unknown), o (OS specific), E (exclude),
R (retain), D (mbind), l (large), p (processor specific)

Elf file type is DYN (Shared object file)

Entry point 0x42104

There are 3 program headers, starting at offset 64

0x40

Program Headers:

Type	Offset	VirtAddr	PhysAddr
	FileSiz	MemSiz	Flags Align
<u>PHDR</u>	0x0000000000000040	0x00000000000042040	0x00000000000042040
	0x00000000000000a8	0x00000000000000a8	R 0x1
INTERP	0x00000000000000e8	0x000000000000420e8	0x000000000000420e8
	0x000000000000001c	0x000000000000001c	R 0x1
	[Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]		
LOAD	0x0000000000000040	0x00000000000042040	0x00000000000042040
	0x00000000000000f6	0x00000000000000f6	R E 0x1000

Relocation section 0 at offset 0x176 contains 1 entry:

Offset	Info	Type	Sym. Value	Sym. Name + Addend
0000000042104	000000000001	R_X86_64_64		cc

```

00000000: 7f45 4c46 0201 0103 0000 0000 0000 0000 .ELF.....
00000010: 0300 3e00 0100 0000 0421 0400 0000 0000 ..>.....!.
00000020: 4000 0000 0000 0000 3601 0000 0000 0000 @.....6....
00000030: 0000 0000 0000 0000 0000 0000 0000 0000 .....@.8...@.
00000040: 0600 0000 0400 0000 4000 0000 0000 0000 .....@.....
00000050: 4020 0400 0000 0000 4020 0400 0000 0000 @.....@.....
00000060: a800 0000 0000 0000 a800 0000 0000 0000 .....@.....
00000070: 0100 0000 0000 0000 0300 0000 0400 0000 .....@.....
00000080: e800 0000 0000 0000 e820 0400 0000 0000 .....@.....
00000090: e820 0400 0000 0000 1c00 0000 0000 0000 .....@.....
000000a0: 1c00 0000 0000 0000 0100 0000 0000 0000 .....@.....
000000b0: 0100 0000 0500 0000 4000 0000 0000 0000 .....@.....
000000c0: 4020 0400 0000 0000 4020 0400 0000 0000 @.....@.....
000000d0: f600 0000 0000 0000 f600 0000 0000 0000 .....@.....
000000e0: 0010 0000 0000 0000 2f6c 6962 3634 2f6c ...../lib64/l
000000f0: 642d 6c69 6e73 7878 362d 3634 2f6c 6e73 d-linux-x86-64.s
00000100: 6f2e 3200 b801 0000 00bf 0100 0000 488d o.2.....H.
00000110: 3513 0000 00ba 0e00 0000 0f05 b83c 0000 5.....<..
00000120: 00bf 0000 0000 0f05 4865 6c6c 6f2c 2077 .....Hello, w
00000130: 6f72 6c64 210a 0000 0000 0000 0000 0000 orld!.....
00000140: 0000 0000 0000 0000 0000 0000 0000 7601 .....v.
00000150: 0000 0000 0000 1800 0000 0000 0000 0000 .....
00000160: 0000 0000 0000 0000 0000 0000 0000 1800 .....
00000170: 0000 0000 0000 0421 0400 0000 0000 0100 .....!.
00000180: 0000 0000 0000 cc00 0000 0000 0000 .....

```

PHDRS CODE

PT_LOAD

```
rela (git: main) $ ./relocation_base  
Segmentation fault (core dumped) ./relocation_base
```

```
rela (git: main) $ valgrind ./relocation_base
=150410= Memcheck, a memory error detector
=150410= Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
=150410= Using Valgrind-3.25.1 and LibVEX; rerun with -h for copyright info
=150410= Command: ./relocation_base
=150410=
=150410= Invalid read of size 8
=150410=   at 0x4021F5A: audit_list_add_dynamic_tag (rtld.c:224)
=150410=   by 0x4021F5A: dl_main (rtld.c:1766)
=150410=   by 0x401ED95: _dl_sysdep_start (dl-sysdep.c:140)
=150410=   by 0x40204C0: _dl_start_final (rtld.c:496)
=150410=   by 0x40204C0: _dl_start (rtld.c:581)
=150410=   by 0x401F307: ??? (in /usr/lib/ld-linux-x86-64.so.2)
=150410= Address 0x8 is not stack'd, malloc'd or (recently) free'd
=150410=
=150410=
=150410= Process terminating with default action of signal 11 (SIGSEGV): dumping core
=150410= Access not within mapped region at address 0x8
=150410=   at 0x4021F5A: audit_list_add_dynamic_tag (rtld.c:224)
=150410=   by 0x4021F5A: dl_main (rtld.c:1766)
=150410=   by 0x401ED95: _dl_sysdep_start (dl-sysdep.c:140)
=150410=   by 0x40204C0: _dl_start_final (rtld.c:496)
=150410=   by 0x40204C0: _dl_start (rtld.c:581)
=150410=   by 0x401F307: ??? (in /usr/lib/ld-linux-x86-64.so.2)
=150410= If you believe this happened as a result of a stack
=150410= overflow in your program's main thread (unlikely but
=150410= possible), you can try to increase the size of the
=150410= main thread stack using the --main-stacksize= flag.
=150410= The main thread stack size used in this run was 8388608.
=150410=
=150410= HEAP SUMMARY:
=150410=   in use at exit: 0 bytes in 0 blocks
=150410=   total heap usage: 0 allocs, 0 frees, 0 bytes allocated
=150410=
=150410= All heap blocks were freed -- no leaks are possible
=150410=
=150410= For lists of detected and suppressed errors, rerun with: -s
=150410= ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
Segmentation fault (core dumped) valgrind ./relocation_base
```



```
219 static void
220 audit_list_add_dynamic_tag (struct audit_list *list, struct link_map *main_map,
221                             unsigned int tag)
222 {
223     ElfW(Dyn) *info = main_map->l_info[ADDRIDX (tag)];
224     const char *strtab = (const char *) D_PTR (main_map, l_info[DT_STRTAB]);
225     if (info != NULL)
226         audit_list_add_string (list, strtab + info->d_un.d_val);
227 }
```

PT_DYNAMIC

Dynamic Section

If an object file participates in dynamic linking, its program header table will have an element of type `PT_DYNAMIC`. This segment contains the `.dynamic` section. A special symbol, `_DYNAMIC`, labels the section, which contains an array of the following structures, defined in `sys/link.h`:

```
typedef struct {
    Elf32_Sword d_tag;
    union {
        Elf32_Word    d_val;
        Elf32_Addr    d_ptr;
        Elf32_Off     d_off;
    } d_un;
} Elf32_Dyn;

typedef struct {
    Elf64_Xword d_tag;
    union {
        Elf64_Xword    d_val;
        Elf64_Addr    d_ptr;
    } d_un;
} Elf64_Dyn;
```

← x64

DT_STRTAB

The address of the string table. Symbol names, dependency names, and other strings required by the runtime linker reside in this table. See ["String Table"](#).

String Table

String table sections hold null-terminated character sequences, commonly called strings. The object file uses these strings to represent symbol and section names. You reference a string as an index into the string table section.

The first byte, which is index zero, holds a null character. Likewise, a string table's last byte holds a null character, ensuring null termination for all strings. A string whose index is zero specifies either no name or a null name, depending on the context.

An empty string table section is permitted. The section header's `sh_size` member will contain zero.

" \x00 " ~~00~~

Construindo do Zero

code

- adicionar a sessão SHT_STRTAB
- adicionar o segmento PT_DYNAMIC
 - adicionar a tag dinâmica DT_STRTAB

[0]

```
rela (git: main) $ readelf -lSrd ./relocation_base
```

There are 2 section headers, starting at offset 0x18e:

Section Headers:

[Nr]	Name	Type	Address	Offset
	Size	EntSize	Flags Link Info Align	
[0]	<no-strings>	STRTAB	0000000000000000	0000020e
	0000000000000001	0000000000000000	0 0	0
[1]	<no-strings>	RELA	0000000000000000	0000020f
	0000000000000018	0000000000000018	0 0	0

Key to Flags:

W (write), A (alloc), X (execute), M (merge), S (strings), I (info),
L (link order), O (extra OS processing required), G (group), T (TLS),
C (compressed), x (unknown), o (OS specific), E (exclude),
R (retain), D (mbind), l (large), p (processor specific)

Elf file type is DYN (Shared object file)

Entry point 0x4215c

There are 4 program headers, starting at offset 64

Program Headers:

Type	Offset	VirtAddr	PhysAddr
	FileSiz	MemSiz	Flags Align
PHDR	0x0000000000000040	0x00000000000042040	0x00000000000042040
	0x00000000000000e0	0x00000000000000e0	R 0x1
INTERP	0x0000000000000120	0x00000000000042120	0x00000000000042120
	0x000000000000001c	0x000000000000001c	R 0x1
[Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]			
DYNAMIC	0x000000000000013c	0x0000000000004213c	0x0000000000004213c
	0x0000000000000020	0x0000000000000020	R 0x1

PT_DYNAMIC

readelf: Error: no .dynamic section in the dynamic segment

LOAD	0x0000000000000040	0x00000000000042040	0x00000000000042040
	0x000000000000014e	0x000000000000014e	R E 0x1000

Dynamic section at offset 0x13c contains 2 entries:

Tag	Type	Name/Value
0x0000000000000005	STRTAB	0x4220e
0x0000000000000000	(NULL)	0x0

Relocation section 0 at offset 0x20f contains 1 entry:

Offset	Info	Type	Sym. Value	Sym. Name + Addend
00000004215c	000000000001	R_X86_64_64		cc

```
rela (git: main) $ ./relocation_base  
Segmentation fault (core dumped) ./relocation_base
```

```
[139] rela (git: main) $ valgrind ./relocation_base
==154993== Memcheck, a memory error detector
==154993== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==154993== Using Valgrind-3.25.1 and LibVEX; rerun with -h for copyright info
==154993== Command: ./relocation_base
==154993==
./relocation_base: symbol lookup error: /usr/lib/valgrind/vgpreload_memcheck-amd64-linux.so: undefined symbol: getpagesize
==154993==
==154993== HEAP SUMMARY:
==154993==       in use at exit: 0 bytes in 0 blocks
==154993==    total heap usage: 0 allocs, 0 frees, 0 bytes allocated
==154993==
==154993== All heap blocks were freed -- no leaks are possible
==154993==
==154993== For lists of detected and suppressed errors, rerun with: -s
==154993== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

```
(gdb) r
Starting program: /home/fukuda/Repos/fnknda/study/obfuscation/rela/relocation_base

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.archlinux.org>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.

Program received signal SIGSEGV, Segmentation fault.
0x00007ffff7fd0b3b in elf_dynamic_do_Rela (map=0x7ffff7ffe2e0, scope=<optimized out>, reladdr=0, relsize=0, nrelative=0, lazy=0, skip_ifunc=<optimized out>)
    at /usr/src/debug/glibc/glibc/elf/do-rel.h:53
53      _ = (const void *) D_PTR (map, l_info[DT_SYMTAB]);
```

DT_SYMTAB

Symbol Table

An object file's symbol table holds information needed to locate and relocate a program's symbolic definitions and references. A symbol table index is a subscript into this array. Index 0 both designates the first entry in the table and serves as the undefined symbol index. See [Table 7-22](#).

Table 7-22 ELF Symbol Table Entry: Index 0

Name	Value	Note
st_name	0	No name
st_value	0	Zero value
st_size	0	No size
st_info	0	No type, local binding
st_other	0	
st_shndx	SHN_UNDEF	No section

Construindo do Zero

code

undef

- adicionar a sessão SHT_SYMTAB
- adicionar a tag dinâmica DT_SYMTAB

```
rela (git: main) $ readelf -Sds ./relocation_base
There are 3 section headers, starting at offset 0x19e:
```

Section Headers:

[Nr]	Name	Type	Address	Offset
	Size	EntSize	Flags Link Info	Align
[0]		STRTAB	0000000000000000	0000025e
	000000000000000001	000000000000000000	0 0	0
readelf: Warning: [1]: Link field (0) should index a string section.				
[1]		SYMTAB	0000000000000000	0000025f
	000000000000000018	000000000000000018	0 0	0
[2]		RELA	0000000000000000	00000277
	000000000000000018	000000000000000018	0 0	0

Key to Flags:

W (write), A (alloc), X (execute), M (merge), S (strings), I (info),
L (link order), O (extra OS processing required), G (group), T (TLS),
C (compressed), x (unknown), o (OS specific), E (exclude),
R (retain), D (mbind), l (large), p (processor specific)

readelf: Error: no .dynamic section in the dynamic segment

Dynamic section at offset 0x13c contains 3 entries:

Tag	Type	Name/Value
0x0000000000000005 (STRTAB)		0x4225e
0x0000000000000006 (SYMTAB)		0x4225f
0x0000000000000000 (NULL)		0x0

UNDEF

Symbol table '' contains 1 entry:

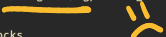
Num:	Value	Size	Type	Bind	Vis	Ndx	Name
→ 0:	0000000000000000	0	NOTYPE	LOCAL	DEFAULT	UND	

readelf: Warning: local symbol 0 found at index ≥ 's sh_info value of 0

```
rela (git: main) $ ./relocation_base  
Hello, world!
```



```
rela (git: main) $ valgrind ./relocation_base
==159209== Memcheck, a memory error detector
==159209== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==159209== Using Valgrind-3.25.1 and LibVEX; rerun with -h for copyright info
==159209== Command: ./relocation_base
==159209==
./relocation_base: symbol lookup error: /usr/lib/valgrind/vgpreload_memcheck-amd64-linux.so: undefined symbol: getpagesize
==159209==
==159209== HEAP SUMMARY:
==159209==       in use at exit: 0 bytes in 0 blocks
==159209==    total heap usage: 0 allocs, 0 frees, 0 bytes allocated
==159209==
==159209== All heap blocks were freed -- no leaks are possible
==159209==
==159209== For lists of detected and suppressed errors, rerun with: -s
==159209== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```



Construindo do Zero

code

- arrumar as tags dinâmicas até rodar

→ DT_RELA

CODIGO ORIGINAL ✓
RELOCAÇÃO X

```
rela (git: main) $ readelf -Sds ./relocation_base
There are 3 section headers, starting at offset 0x1ae:
```

Section Headers:

[Nr]	Name	Type	Address	Offset
	Size	EntSize	Flags Link Info	Align
[0]		STRTAB	0000000000000000	0000026e
	0000000000000001	0000000000000000	0 0	0
readelf: Warning: [1]: Link field (0) should index a string section.				
[1]		SYMTAB	0000000000000000	0000026f
	0000000000000018	0000000000000018	0 0	0
[2]		RELA	0000000000000000	00000287
	0000000000000018	0000000000000018	0 0	0

Key to Flags:

W (write), A (alloc), X (execute), M (merge), S (strings), I (info),
L (link order), O (extra OS processing required), G (group), T (TLS),
C (compressed), x (unknown), o (OS specific), E (exclude),
R (retain), D (mbind), l (large), p (processor specific)
readelf: Error: no .dynamic section in the dynamic segment

Dynamic section at offset 0x13c contains 4 entries:

Tag	Type	Name/Value
0x0000000000000005	(STRTAB)	0x4226e
0x0000000000000006	(SYMTAB)	0x4226f
0x0000000000000007	(RELA)	0x42287
0x0000000000000000	(NULL)	0x0

Symbol table '' contains 1 entry:

Num:	Value	Size	Type	Bind	Vis	Ndx	Name
0:	0000000000000000	0	NOTYPE	LOCAL	DEFAULT	UND	

readelf: Warning: local symbol 0 found at index ≥ 's sh_info value of 0

```
rela (git: main) $ ./relocation_base  
Segmentation fault (core dumped) ./relocation_base
```

```
(gdb) r
Starting program: /home/fukuda/Repos/fnknda/study/obfuscation/rela/relocation_base

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.archlinux.org>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.

Program received signal SIGSEGV, Segmentation fault.
0x00007ffff7fe2242 in elf_get_dynamic_info (l=0x7ffff7ffe2e0, bootstrap=false, static_pie_bootstrap=false) at ./get-dynamic-info.h:123
123      assert (info[DT_RELAENT] -> d_un.d_val == sizeof (ElfW(Rela)));
```

DT_RELAENT

```
rela (git: main) $ readelf -d ./relocation_base
readelf: Error: no .dynamic section in the dynamic segment
```

Dynamic section at offset 0x13c contains 5 entries:

Tag	Type	Name/Value
0x0000000000000005	(STRTAB)	0x4227e
0x0000000000000006	(SYMTAB)	0x4227f
0x0000000000000007	(RELA)	0x42297
0x0000000000000009	(RELAENT)	24 (bytes)
0x0000000000000000	(NULL)	0x0

```
rela (git: main) $ ./relocation_base  
Segmentation fault (core dumped) ./relocation_base
```



```
(gdb) r
Starting program: /home/fukuda/Repos/fnknda/study/obfuscation/rela/relocation_base

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.archlinux.org>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.

Program received signal SIGSEGV, Segmentation fault.
_dl_relocate_object_no_relo (l=@entry=0x7ffff7ffe2e0, scope=<optimized out>, reloc_mode=<optimized out>, consider_profiling=0) at dl-reloc.c:296
296 _ _ ELF_DYNAMIC_RELOCATE (l, scope, lazy, consider_profiling, skip_ifunc);
```

?

```
191 # define ELF_DYNAMIC_RELOCATE(map, scope, lazy, consider_profile, skip_ifunc) \
192 do { ELF_DYNAMIC_RELOCATE \
193     int edr_lazy = elf_machine_runtime_setup ((map), (scope), (lazy), \
194                                             (consider_profile)); \
195     if (!is_rtld_link_map (map) || DO_RTLD_BOOTSTRAP) \
196         ELF_DYNAMIC_DO_RELR (map); \
197     ELF_DYNAMIC_DO_REL ((map), (scope), edr_lazy, skip_ifunc); \
198     ELF_DYNAMIC_DO_RELA ((map), (scope), edr_lazy, skip_ifunc); \
199     ELF_DYNAMIC_AFTER_RELOC ((map), (edr_lazy)); \
200 } while (0)
```




```
147 # define ELF_DYNAMIC_DO_RELA(map, scope, lazy, skip_ifunc) \
148 _ELF_DYNAMIC_DO_RELOC (RELA, Rela, map, scope, lazy, skip_ifunc, _ELF_CHECK_REL)
```

RELA

```
81 # define _ELF_DYNAMIC_DO_RELOC(RELOC, reloc, map, scope, do_lazy, skip_ifunc, test_rel) \
82 do { \
83     struct { ElfW(Addr) start, size; \
84         __typeof (((ElfW(Dyn) *) 0)->d_un.d_val) nrelative; int lazy; } \
85     ranges[2] = { { 0, 0, 0, 0 }, { 0, 0, 0, 0 } }; \
86 \
87     /* With DT_RELR, DT_RELA/DT_REL can have zero value. */ \
88     if ((map)->l_info[DT_#RELOC] != NULL \
89         && (map)->l_info[DT_#RELOC]->d_un.d_ptr != 0) \
90     { \
91         ranges[0].start = D_PTR((map), l_info[DT_#RELOC]); \
92         ranges[0].size = (map)->l_info[DT_#RELOC#SZ]->d_un.d_val; \
93         if (map->l_info[VERSYMIDX(DT_#RELOC#COUNT)] != NULL) \
94             ranges[0].nrelative \
95                 = map->l_info[VERSYMIDX(DT_#RELOC#COUNT)]->d_un.d_val; \
96     } \
97     if ((map)->l_info[DT_#RELOC] != NULL \
```

DT_RELA

DT_RELASZ

DT_RELACOUNT



```
rela (git: main) $ readelf -ld ./relocation_base
```

Elf file type is DYN (Shared object file)

Entry point 0x4219c

There are 4 program headers, starting at offset 64

Program Headers:

Type	Offset FileSiz	VirtAddr MemSiz	PhysAddr Flags Align
PHDR	0x0000000000000040 0x00000000000000e0	0x00000000000042040 0x00000000000000e0	0x00000000000042040 R 0x1
INTERP	0x0000000000000120 0x00000000000001c	0x00000000000042120 0x00000000000001c	0x00000000000042120 R 0x1
[Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]			
DYNAMIC	0x000000000000013c 0x0000000000000060	0x0000000000004213c 0x0000000000000060	0x0000000000004213c R 0x1
readelf: Error: no .dynamic section in the dynamic segment			
LOAD	0x0000000000000040 0x000000000000027f	0x00000000000042040 0x000000000000027f	0x00000000000042040 R E 0x1000

Section to Segment mapping:

Segment Sections ...

00
01
02
03

READ + EXECUTE

Dynamic section at offset 0x13c contains 6 entries:

Tag	Type	Name/Value
0x0000000000000005	(STRTAB)	0x4228e
0x0000000000000006	(SYMTAB)	0x4228f
0x0000000000000007	(RELA)	0x422a7
0x0000000000000009	(RELAENT)	24 (bytes)
0x0000000000000008	(RELASZ)	24 (bytes)
0x0000000000000000	(NULL)	0x0

```
rela (git: main) $ ./relocation_base  
Segmentation fault (core dumped) ./relocation_base
```

```
(gdb) r
Starting program: /home/fukuda/Repos/fnknda/study/obfuscation/rela/relocation_base

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.archlinux.org>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.

Program received signal SIGSEGV, Segmentation fault.
elf_machine_rela (map=<optimized out>, scope=<optimized out>, reloc=0x5555555532a7, sym=0x55555555328f, version=0x0, reloc_addr_arg=<optimized out>,
    skip_ifunc=<optimized out>) at ../sysdeps/x86_64/dl-machine.h:318
318     *reloc_addr = value;
```

? permissions

Construindo do Zero

code

- arrumar o PT_LOAD → PF_W
(WRITE)

```
rela (git: main) $ readelf -l ./relocation_base
```

Elf file type is DYN (Shared object file)

Entry point 0x4219c

There are 4 program headers, starting at offset 64

Program Headers:

Type	Offset FileSiz	VirtAddr MemSiz	PhysAddr Flags Align
PHDR	0x0000000000000040	0x00000000000042040	0x00000000000042040
	0x00000000000000e0	0x00000000000000e0	R 0x1
INTERP	0x0000000000000120	0x00000000000042120	0x00000000000042120
	0x000000000000001c	0x000000000000001c	R 0x1
[Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]			
DYNAMIC	0x000000000000013c	0x0000000000004213c	0x0000000000004213c
	0x0000000000000060	0x0000000000000060	R 0x1
readelf: Error: no .dynamic section in the dynamic segment			
LOAD	0x0000000000000040	0x00000000000042040	0x00000000000042040
	0x000000000000018e	0x000000000000018e	RWE 0x1000

Section to Segment mapping:

Segment Sections ...

00
01
02
03

=
READ + WRITE + EXECUTE

```
rela (git: main) $ ./relocation_base  
Trace/breakpoint trap (core dumped) ./relocation_base
```



O Arquivo Final

file

EHDR dynamic

PHDR PT_PHDR
PT_INTERP
PT_DYNAMIC
PT_LOAD

SHDR SHT_STRTAB
SHT_SYMTAB
SHT_RELA

Resultados

Pontos Fortes

gdb

- método incomum de ofuscação de entrypoint
- depende somente de ferramentas nativas →
- logica de decoding externa e legitima → runtime linker
- dificulta análise dinâmica

INT3

Pontos Fracos

gdb

- permissão de mapeamento
- tipos de relocação → $\cdot 64$
 $\cdot SIZE64$
- espaço de relocação limitado
- muito trabalho para infectar um arquivo

OFFSETS

Detecção[6][7]

md5sum

- mapas RWE
- alinhamento
- intersecções
- tipos de relocação

SIZE64

Referências i

Idd

- [1] Netspooky. *ELF Binary Mangling Series*. URL: <https://n0.101/ebm/>.
- [2] sbliip. *Implementing the PT_NOTE Infection Method in x64 Assembly*. URL: <https://tmpout.sh/1/2.html>.
- [3] s01den. *A short note on entrypoint obscuring in ELF binaries*. URL: <https://tmpout.sh/2/2.html>.
- [4] SOS4. *Relocation Revelation: Unpacking the Secrets of ELF*. URL: <https://tmpout.sh/4/4.html>.
- [5] GNU C Library master sources. *sourceware.org Git - glibc.git/blob - elf/elf.h*. URL: <https://sourceware.org/git/?p=glibc.git;a=blob;f=elf/elf.h;h=46a01281cb0fb5322d5124f0443c11dea4d5b721;hb=f762ccf84f122d1354f103a151cba8bde797d521>.

Referências ii

Idd

← ORIGINAL

- [6] ulexec. *elf_rela_obfuscation*. URL: https://github.com/ulexec/elf_rela_obfuscation.
- [7] peding. *relobfuscate*. URL: ← COMPLETO <https://github.com/peding/relobfuscate>.

Perguntas?

Fim
exit

Muito obrigado :):